

PLC Calculator

by

Tatiya Seehatrakul

A Project Report Submitted in
AT70.07 Programming Languages and Compilers

Instructor: Prof. Phan Minh Dung

Teacher Assistant: Akraradet Sinsamersuk

Asian Institute of Technology
School of Engineering and Technology
Thailand
March 2025

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to Professor Phan Minh Dung for his insightful guidance, continuous encouragement, and unwavering support throughout the AT70.07 Programming Languages and Compilers course. His expertise and mentorship have been instrumental in shaping both my understanding and execution of this project.

I am also sincerely thankful to Mr. Akraradet Sinsamersuk, our dedicated teaching assistant, whose patience, timely feedback, and technical support greatly helped me navigate the challenges faced during development.

Finally, I would like to extend my appreciation to my fellow classmates for their collaboration, shared ideas, and motivating spirit, all of which played a meaningful role in the successful completion of this Prefix Calculator project.

ABSTRACT

This project successfully implements a Prefix Expression Calculator that converts prefix expressions into infix notation, evaluates the result, and presents the output through a graphical user interface built using PyQt6. The system currently supports two arithmetic operations: addition and multiplication, and accepts both positive and negative integers as input.

The core functionality is powered by a single custom `PrefixParser` developed using the SLY (Simple Lex & Yacc) library. The `PrefixParser` handles both the evaluation of expressions and the construction of their corresponding infix representation. By managing these tasks within a unified parser, the system remains lightweight and efficient while maintaining clarity in its design.

The overall result is a functional and user-friendly calculator that highlights key concepts of parsing and GUI development. Future improvements may include expanding operator support, enhancing input validation, and adding more interactive features to the interface.

CONTENTS

	Page
ACKNOWLEDGEMENTS	2
ABSTRACT	3
CHAPTER 1 INTRODUCTION	5
CHAPTER 2 METHODOLOGY	6
2.1 System Overview	6
2.2 Lexer Design	6
2.3 Parser Design	7
2.3.1 Grammar Rules	7
2.3.2 First and Follow Sets	7
2.3.3 Closure Table	7
2.3.4 SLR Parsing Table	9
2.4 GUI Design	10
2.4.1 GUI Components	10
CHAPTER 3 REPOSITORY	11
CHAPTER 4 CONCLUSION	12

CHAPTER 1

INTRODUCTION

Mathematical expressions can be represented in three common notations: Infix, Prefix, and Postfix. Infix notation places the operator between operands (e.g., $2 + 3$), which is most familiar in standard arithmetic. Prefix notation (e.g., $+ 2 3$) places the operator before its operands, while Postfix notation (e.g., $2 3 +$) places the operator after the operands.

This project implements a calculator that:

- Accepts a prefix expression as input.
- Converts it into infix notation.
- Computes and returns the final result.
- Support positive and negative integers.
- Displays all outputs in a GUI using PyQt6.

The system is developed using Python and the SLY (Simple Lex & Yacc) library for parsing and evaluation, ensuring a structured and extensible parsing mechanism.

CHAPTER 2

METHODOLOGY

2.1 System Overview

The system is modular and comprises the following main components:

- MyLexer: Uses the SLY library to tokenize the input arithmetic expressions.
- PrefixParser: Converts prefix expressions to infix while performing evaluation.
- Graphical User Interface (GUI): Built with PyQt6 to support user interaction.

2.2 Lexer Design

The MyLexer class defines tokens for all supported operators, parentheses, numbers, and assignment symbols. To recognize integers (both positive and negative), the lexer uses the regular expression `@_(r'-?\d+')` for number tokenization.

This regular expression works as follows:

- `r' . . . '`: Defines a raw string in Python, treating backslashes as literal characters.
- `-?`: Matches an optional minus sign, allowing both positive and negative integers.
- `\d+`: Matches one or more digits, where `\d` represents digits from 0 to 9.

2.3 Parser Design

For this project, the chosen parsing technique is the SLR(1) parser, a type of bottom-up parser based on LR parsing with a simplified lookahead mechanism. This parser is suitable for handling arithmetic expressions written in prefix form.

2.3.1 Grammar Rules

The grammar used in this calculator project supports basic arithmetic operations in prefix notation. The rules are defined as follows:

- $S \rightarrow E$
- $E \rightarrow + E E$
- $E \rightarrow * E E$
- $E \rightarrow \text{number}$

2.3.2 First and Follow Sets

The computed FIRST and FOLLOW sets for the non-terminal symbols are:

Nonterminal	FIRST	FOLLOW
S	{ +, *, number }	{ \$ }
E	{ +, *, number }	{ \$, +, * }

2.3.3 Closure Table

State 0
$S \rightarrow \cdot E$
$E \rightarrow \cdot + E E$
$E \rightarrow \cdot * E E$
$E \rightarrow \cdot \text{number}$

State 1
$S \rightarrow E \cdot$

State 2
$E \rightarrow + \cdot E E$
$E \rightarrow \cdot + E E$
$E \rightarrow \cdot * E E$
$E \rightarrow \cdot \text{number}$

State 3
$E \rightarrow * \cdot E E$
$E \rightarrow \cdot + E E$
$E \rightarrow \cdot * E E$
$E \rightarrow \cdot \text{number}$

State 4
$E \rightarrow \text{number} \cdot$

State 5
$E \rightarrow + E \cdot E$
$E \rightarrow \cdot + E E$
$E \rightarrow \cdot * E E$
$E \rightarrow \cdot \text{number}$

State 6
$E \rightarrow * E \cdot E$
$E \rightarrow \cdot + E E$
$E \rightarrow \cdot * E E$
$E \rightarrow \cdot \text{number}$

State 7
$E \rightarrow + E E \cdot$

State 8
$E \rightarrow * E E \cdot$

2.3.4 SLR Parsing Table

The SLR parsing table generated from the grammar is as follows:

State	ACTION				GOTO	
	+	*	number	\$	S	E
0	s2	s3	s4			1
1				acc		
2	s2	s3	s4			5
3	s2	s3	s4			6
4	r3	r3	r3	r3		
5	s2	s3	s4			7
6	r1	r1	r1	r1		
7	r2	r2	r2	r2		

2.4 GUI Design

The GUI, built with PyQt6 and designed in Qt Designer, accepts prefix expressions and supports two operators: + and *. It converts inputs to infix, evaluates the result, and displays both the infix form and result. The interface features a clean layout with color-coded buttons and hover effects for better interactivity.

2.4.1 GUI Components

The interface contains the following major components:

- **Input Field:** A QLineEdit named 'input_prefix' for prefix input.
- **Infix Output:** A read-only QLineEdit named 'output_infix' for infix output.
- **Answer Output:** A QLCDNumber widget named 'output_answer' for results.
- **Digit Buttons:** Buttons 0–9, labeled 'button_0' to 'button_9' for integers.
- **Operator Buttons:** QPushButtons for '+', '*' named 'button_plus' and 'button_star'.
- **Equal Button:** QPushButtons labeled '=' named 'button_equal'.
- **Clear Button:** QPushButtons labeled 'C' named 'button_clear'.
- **Labels:** QLabel widgets used to identify all input and output fields.
- **Container Frame:** A QFrame named 'frame' to organize the layout.

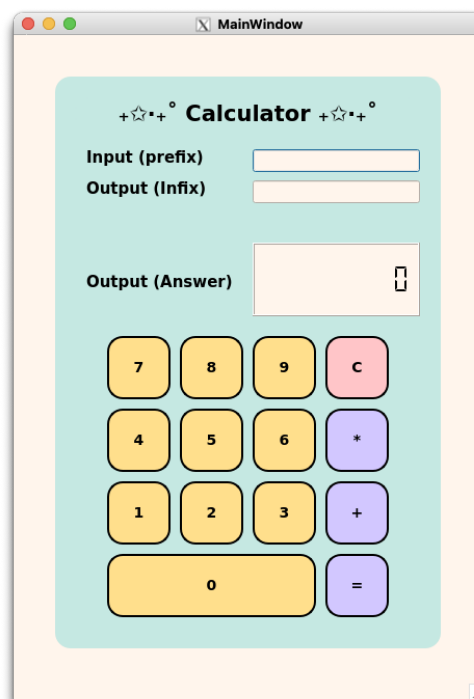


Figure 2.1

Graphical User Interface of the Prefix Expression Calculator

CHAPTER 3

REPOSITORY

5.1.1 GitHub Repository

The GitHub repository contains all source code, configuration files, and related resources for the project.



Figure 3.1 *PLC Calculator*

CHAPTER 4

CONCLUSION

This project successfully implements a Prefix Expression Calculator that converts prefix expressions into infix notation, evaluates the result, and presents the output through a graphical user interface built using PyQt6. The system currently supports two arithmetic operations: addition and multiplication, and accepts both positive and negative integers as input.

The core functionality is powered by a single custom `PrefixParser` developed using the `SLY` (Simple Lex & Yacc) library. The `PrefixParser` handles both the evaluation of expressions and the construction of their corresponding infix representation. By managing these tasks within a unified parser, the system remains lightweight and efficient while maintaining clarity in its design.

The overall result is a functional and user-friendly calculator that highlights key concepts of parsing and GUI development. Future improvements may include expanding operator support, enhancing input validation, and adding more interactive features to the interface.